Développement d’applications mobiles

Guide de référence : sécurité, clean code et architecture

Android (Kotlin) · iOS (Swift) · Flutter · React Native

Préparé pour Faycal Chefai — Trois-Rivières (Québec)
Version du 10 octobre 2026

Méthode — Ce guide s’appuie uniquement sur des sources officielles ou reconnues (OWASP, NIST, IETF, Android Developers, Apple Developer, Flutter, React Native, Commission d’accès à l’information du Québec, Commissariat à la protection de la vie privée du Canada) consultées le 10 octobre 2026. Les liens sont regroupés dans la bibliographie (section 6). Ce document ne constitue pas un avis juridique : pour la conformité à la Loi 25, validez avec un ou une juriste.

Table des matières

1. Introduction et façon d’utiliser ce guide

2. Sécurité mobile (OWASP MASVS v2, Mobile Top 10 2024, API Top 10 2023)

   2.1 Référentiels · 2.2 MASVS catégorie par catégorie (STORAGE → PRIVACY)

   2.3 Backend/API · 2.4 Secrets · 2.5 OAuth2/OIDC + PKCE et jetons

   2.6 Chaîne d’approvisionnement et CI/CD · 2.7 Tests de sécurité · 2.8 Vie privée au Québec

   2.9 Checklist de sécurité consolidée

3. Clean code

4. Architecture (MVVM, MVI, Clean Architecture, modulaire) et recommandation par stack

5. Cycle de développement complet (planification → STRIDE → CI/CD → stores → monitoring)

6. Bibliographie et liens

Annexe A. Correspondance MASVS ↔ OWASP Mobile Top 10 2024

1. Introduction et façon d’utiliser ce guide

Ce guide s’adresse à un développeur web qui prépare sa première application mobile sans avoir encore arrêté la stack. Il couvre donc les quatre options les plus courantes : **Android natif (Kotlin, Jetpack Compose)**, **iOS natif (Swift, SwiftUI)**, **Flutter (Dart)** et **React Native (TypeScript)**. Les principes sont communs ; seuls les mécanismes d’implémentation changent.

Trois idées structurent le document :

- **Le client mobile n’est pas une zone de confiance.** Tout ce qui est livré dans l’application (code, ressources, clés, logique de contrôle) peut être lu, modifié ou contourné par une personne qui contrôle l’appareil. La sécurité réelle se décide côté serveur ; l’application ajoute des couches de défense en profondeur.
- **Les plateformes fournissent déjà l’essentiel.** Keystore/Keychain, TLS par défaut (Network Security Config, App Transport Security), biométrie liée à la cryptographie, attestation (Play Integrity, App Attest) : utilisez-les plutôt que de réinventer.
- **La qualité du code et l’architecture sont des mesures de sécurité.** Un code lisible, testé, découpé en couches et revu par les pairs réduit les défauts, donc les vulnérabilités, et accélère les correctifs.

Conventions : les exigences sont formulées en impératif ; « MASVS-XXX-n » renvoie aux contrôles officiels de l’OWASP MASVS v2.1.0 ; les cases ☐ peuvent être cochées lors d’une revue. Les extraits de code sont volontairement courts et illustratifs : vérifiez toujours la version courante des API dans la documentation officielle avant de les utiliser en production.

2. Sécurité mobile

2.1 Les référentiels à connaître

Référentiel	Rôle	Version / date
OWASP MASVS	Standard de vérification : liste les contrôles qu'une app sécurisée doit satisfaire, répartis en 8 groupes (STORAGE, CRYPTO, AUTH, NETWORK, PLATFORM, CODE, RESILIENCE, PRIVACY).	v2.1.0 (janvier 2024)
OWASP MASTG	Guide de tests : techniques, outils et tests concrets (Android/iOS) pour vérifier les contrôles MASVS.	v2.0.0 publiée en juin 2026 (GitHub)
OWASP MASWE	Énumération des faiblesses mobiles qui relie MASVS et MASTG.	Projet MAS
OWASP Mobile Top 10	Sensibilisation : les 10 risques mobiles les plus critiques (M1 à M10).	Édition 2024 (version finale)
OWASP API Security Top 10	Les 10 risques majeurs des API — votre backend mobile.	Édition 2023
NIST SP 800-163 Rev. 1	Processus de vérification (« vetting ») de la sécurité des applications mobiles.	Avril 2019
NIST SP 800-63B-4	Authentification et gestion des authentificateurs (mots de passe, MFA, phishing-resistance).	Juillet 2025 (remplace 800-63B de 2017/2020)
RFC 8252 (BCP 212)	OAuth 2.0 pour les applications natives : navigateur externe obligatoire, redirections, PKCE.	Octobre 2017
RFC 7636	PKCE — protection du code d'autorisation (méthode S256).	Septembre 2015
RFC 9700 (BCP 240)	Bonnes pratiques de sécurité OAuth 2.0 (mise à jour de la menace).	Janvier 2025

OWASP Mobile Top 10 (édition 2024) : **M1** Improper Credential Usage · **M2** Inadequate Supply Chain Security · **M3** Insecure Authentication/Authorization · **M4** Insufficient Input/Output Validation · **M5** Insecure Communication · **M6** Inadequate Privacy Controls · **M7** Insufficient Binary Protections · **M8** Security Misconfiguration · **M9** Insecure Data Storage · **M10** Insufficient Cryptography.

À retenir — Le MASVS v2 n'intègre plus les niveaux L1/L2/R de la v1 : le projet MAS les a transformés en « profils de test » (MAS-L1 sécurité essentielle, MAS-L2 sécurité avancée, MAS-R résilience, MAS-P confidentialité de base), qui déterminent les contrôles et tests applicables selon le modèle de menace. Pour une première app grand public sans données très sensibles, visez au minimum tous les contrôles STORAGE, CRYPTO, AUTH, NETWORK-1, PLATFORM, CODE et PRIVACY ; ajoutez NETWORK-2 et RESILIENCE si vous manipulez de l'argent, des données de santé ou une propriété intellectuelle à protéger.

2.2 MASVS catégorie par catégorie

Pour chaque groupe : les contrôles officiels (énoncés traduits librement de l'anglais), les exigences concrètes, des exemples de code, les erreurs fréquentes et une mini-checklist.

2.2.1 MASVS-STORAGE — stockage des données sensibles

Contrôle	Énoncé officiel (traduction)
MASVS-STORAGE-1	L'app stocke les données sensibles de manière sécurisée.
MASVS-STORAGE-2	L'app empêche les fuites de données sensibles.

Exigences concrètes

- Ne stockez localement que ce qui est indispensable ; préférez garder les données sensibles côté serveur et en mémoire.
- Jetons, identifiants et clés : **Keychain** (iOS) et **Android Keystore** (clé matérielle) ; jamais `SharedPreferences`/`UserDefaults`/`AsyncStorage` en clair.
- Fichiers sensibles : stockage interne privé + chiffrement (Android) ; classe de protection `.complete` (iOS Data Protection).
- Excluez les données sensibles des sauvegardes (Android `dataExtractionRules`/`fullBackupContent`, iOS attribut `ThisDeviceOnly` pour la Keychain, exclusion iCloud des fichiers).
- Prévenez les fuites : pas de données personnelles ou de jetons dans les journaux (logs), le presse-papiers, les captures d'écran du sélecteur d'apps, les caches HTTP, les rapports de plantage et les outils d'analytique.

Android / Kotlin — Keystore

```
// Générer une clé AES-256 non exportable dans l'Android Keystore
val spec = KeyGenParameterSpec.Builder(
    "token_key",
    KeyProperties.PURPOSE_ENCRYPT or KeyProperties.PURPOSE_DECRYPT
)
    .setBlockModes(KeyProperties.BLOCK_MODE_GCM)
    .setEncryptionPaddings(KeyProperties.ENCRYPTION_PADDING_NONE)
    .setKeySize(256)
    .build()
val keyGen = KeyGenerator.getInstance(KeyProperties.KEY_ALGORITHM_AES, "AndroidKeyStore")
keyGen.init(spec)
val key = keyGen.generateKey()

val cipher = Cipher.getInstance("AES/GCM/NoPadding")
cipher.init(Cipher.ENCRYPT_MODE, key)        // IV aléatoire fourni par le Keystore
val ciphertext = cipher.doFinal(token.toByteArray())
val iv = cipher.iv                           // à stocker avec le chiffré (non secret)
```

Attention (Android) — La bibliothèque Jetpack `androidx.security:security-crypto` (EncryptedSharedPreferences, EncryptedFile) a vu toutes ses API dépréciées dans sa version stable 1.1.0 et ne recevra plus de versions (documentation Android « Cryptography »). Pour un nouveau projet, utilisez directement l'Android Keystore (comme ci-dessus) avec DataStore ou des fichiers internes, ou une bibliothèque maintenue.

iOS / Swift — Keychain et Data Protection

```
// Enregistrer un jeton dans la Keychain, non migrable vers un autre appareil
let query: [String: Any] = [
    kSecClass as String:            kSecClassGenericPassword,
    kSecAttrService as String:      "com.exemple.app.auth",
    kSecAttrAccount as String:      "refresh_token",
    kSecValueData as String:        Data(token.utf8),
    kSecAttrAccessible as String:   kSecAttrAccessibleWhenUnlockedThisDeviceOnly
]
SecItemDelete(query as CFDictionary)          // remplace l'éventuelle valeur
let status = SecItemAdd(query as CFDictionary, nil)
guard status == errSecSuccess else { throw KeychainError(status) }

// Fichier protégé tant que l'appareil est verrouillé
try data.write(to: url, options: .completeFileProtection)
```

Flutter / Dart

```
// pubspec.yaml : flutter_secure_storage (Keychain sur iOS, Keystore sur Android)
```

```
const storage = FlutterSecureStorage();
await storage.write(key: 'refresh_token', value: token);
final t = await storage.read(key: 'refresh_token');
```

React Native / TypeScript

```
// React Native : react-native-keychain (ou expo-secure-store avec Expo)
import * as Keychain from 'react-native-keychain';
await Keychain.setGenericPassword('refresh_token', token, {
  accessible: Keychain.ACCESSIBLE.WHEN_UNLOCKED_THIS_DEVICE_ONLY,
});
const creds = await Keychain.getGenericPassword();
```

Android — sauvegardes et FLAG_SECURE

```
<!-- AndroidManifest.xml : exclure les données des sauvegardes (Android 12+) -->
<application
    android:dataExtractionRules="@xml/data_extraction_rules"
    android:fullBackupContent="@xml/backup_rules"   <!-- Android 11 et moins -->
    ... >

// Masquer le contenu d'un écran sensible (captures, aperçu du sélecteur d'apps)
window.setFlags(WindowManager.LayoutParams.FLAG_SECURE,
                WindowManager.LayoutParams.FLAG_SECURE)
```

Erreurs fréquentes : jeton dans `AsyncStorage` ou `SharedPreferences` ; état Redux complet persisté avec des données personnelles (mise en garde explicite de la doc React Native) ; `Log.d("token", token)` laissé en production ; base SQLite non chiffrée contenant des données de santé ; fichiers dans le stockage externe partagé ; données sensibles envoyées à Sentry/Crashlytics.

Checklist STORAGE

- ☐ Inventaire des données sensibles stockées sur l'appareil, avec justification.
- ☐ Jetons/secrets uniquement dans Keychain/Keystore (ou wrapper reconnu).
- ☐ Sauvegardes : données sensibles exclues (règles Android, `ThisDeviceOnly` iOS).
- ☐ Aucun secret ni donnée personnelle dans les logs, crash reports, analytics.
- ☐ Écrans sensibles masqués (FLAG_SECURE / vue de remplacement à la mise en arrière-plan iOS).
- ☐ Purge des données locales à la déconnexion et à la suppression du compte.

2.2.2 MASVS-CRYPTO — cryptographie

Contrôle	Énoncé officiel (traduction)
MASVS-CRYPTO-1	L'app emploie une cryptographie actuelle et robuste, utilisée selon les bonnes pratiques de l'industrie.
MASVS-CRYPTO-2	L'app gère les clés selon les bonnes pratiques de l'industrie.

- N'inventez pas de cryptographie. Utilisez les API de la plateforme : JCA + Android Keystore ; CryptoKit / Security framework (Secure Enclave) sur iOS ; sur Flutter/RN, des plugins qui délèguent à ces API natives.
- Algorithmes recommandés par Android Developers : AES-256 en mode GCM (ou CBC) pour le chiffrement, SHA-2 (SHA-256) pour le hachage, HMAC-SHA256, ECDSA avec SHA-256 pour la signature.
- Bannissez : MD5/SHA-1 pour la sécurité, DES/3DES, RC4, AES-ECB, IV fixe ou réutilisé (catastrophique en GCM), `java.util.Random`/`Math.random()` pour des secrets. Utilisez `SecureRandom` / `SecRandomCopyBytes` / `Random.secure()` (Dart).

- Clés : générées sur l'appareil dans le matériel sécurisé (Keystore, StrongBox si disponible, Secure Enclave), non exportables, à usage restreint (but, mode, authentification utilisateur requise si pertinent). Aucune clé symétrique codée en dur.
- Ne spécifiez pas de fournisseur JCA sauf `AndroidKeyStore` (recommandation officielle Android).
- Mots de passe côté serveur : fonction de dérivation lente et salée (Argon2id, scrypt, bcrypt ou PBKDF2) — jamais un simple SHA-256.

iOS / Swift — CryptoKit

```
// iOS : clé P-256 générée et conservée dans la Secure Enclave (CryptoKit)
import CryptoKit
let privateKey = try SecureEnclave.P256.Signing.PrivateKey()
let signature  = try privateKey.signature(for: payload)
// privateKey.dataRepresentation est un blob chiffré utilisable uniquement
// par la Secure Enclave de cet appareil : à stocker dans la Keychain.
```

Erreurs fréquentes : clé AES dans `strings.xml` ou dans une constante Dart ; `Cipher.getInstance("AES")` (mode par défaut potentiellement ECB) ; IV constant ; « chiffrement » Base64 ; hachage maison des mots de passe côté client à la place de TLS.

Checklist CRYPTO

- ☐ Aucun algorithme obsolète (MD5, SHA-1, DES, ECB, RC4) dans le code ni les dépendances utilisées.
- ☐ Mode et padding toujours explicites (`AES/GCM/NoPadding`).
- ☐ IV/nonce aléatoire et unique par chiffrement.
- ☐ Clés générées dans Keystore/Secure Enclave, non exportables, jamais en dur.
- ☐ Générateur aléatoire cryptographique pour tout secret.

2.2.3 MASVS-AUTH — authentification et autorisation

Contrôle	Énoncé officiel (traduction)
MASVS-AUTH-1	L'app utilise des protocoles d'authentification et d'autorisation sécurisés et suit les bonnes pratiques pertinentes.
MASVS-AUTH-2	L'app réalise l'authentification locale de manière sécurisée selon les bonnes pratiques de la plateforme.
MASVS-AUTH-3	L'app protège les opérations sensibles par une authentification supplémentaire.

- Authentification et autorisation **décidées par le serveur**. L'app ne fait que présenter des jetons ; chaque requête est autorisée côté API (voir 2.3).
- Protocoles standard : OAuth 2.0 + OpenID Connect avec code d'autorisation + PKCE via le navigateur système (RFC 8252) — détails en 2.5. Envisagez les **passkeys** (Credential Manager sur Android, Authentication Services sur iOS), résistantes à l'hameçonnage.
- Mots de passe (NIST SP 800-63B-4) : au moins 15 caractères quand le mot de passe est le seul facteur (au moins 8 s'il fait partie d'une MFA) ; accepter au moins 64 caractères ; pas de règles de composition imposées ; pas de changement périodique forcé ; comparaison avec une liste de mots de passe compromis/courants ; limitation du nombre d'essais.
- Biométrie (AUTH-2) : la biométrie déverrouille une **clé** ; elle ne doit pas être un simple booléen « succès/échec » dans le code de l'app, facilement contournable avec un outil d'instrumentation comme Frida.
- Opérations sensibles (AUTH-3) : changement de courriel/mot de passe, paiement, export de données → ré-authentification ou confirmation biométrique liée à une clé, et contrôle côté serveur.

- Sessions : jetons d'accès courts, révocation côté serveur à la déconnexion, au changement de mot de passe et en cas de suspicion de compromission.

Android / Kotlin — BiometricPrompt + CryptoObject

```
// Android : biométrie liée à une clé Keystore (CryptoObject)
val spec = KeyGenParameterSpec.Builder("bio_key",
        KeyProperties.PURPOSE_ENCRYPT or KeyProperties.PURPOSE_DECRYPT)
    .setBlockModes(KeyProperties.BLOCK_MODE_GCM)
    .setEncryptionPaddings(KeyProperties.ENCRYPTION_PADDING_NONE)
    .setUserAuthenticationRequired(true)          // clé inutilisable sans auth
    .setInvalidatedByBiometricEnrollment(true)    // nouvelle empreinte => clé invalidée
    .build()
// ... générer la clé, puis :
cipher.init(Cipher.ENCRYPT_MODE, secretKey)
biometricPrompt.authenticate(promptInfo, BiometricPrompt.CryptoObject(cipher))
// Dans onAuthenticationSucceeded : result.cryptoObject?.cipher?.doFinal(...)
```

iOS / Swift — Keychain + biométrie

```
// iOS : élément Keychain lisible seulement après Face ID / Touch ID
let access = SecAccessControlCreateWithFlags(nil,
    kSecAttrAccessibleWhenPasscodeSetThisDeviceOnly,
    .biometryCurrentSet,            // invalidé si les biométries changent
    nil)!
let query: [String: Any] = [
    kSecClass as String:              kSecClassGenericPassword,
    kSecAttrAccount as String:        "refresh_token",
    kSecAttrAccessControl as String:  access,
    kSecValueData as String:          Data(token.utf8)
]
SecItemAdd(query as CFDictionary, nil)
// Info.plist : NSFaceIDUsageDescription obligatoire pour Face ID
```

En Flutter, le plugin `local_auth` renvoie un booléen : pour un usage sensible, combinez-le avec un stockage protégé par biométrie au niveau natif (par exemple les options d'accès biométrique de `flutter_secure_storage` ou un canal de plateforme), et en React Native utilisez les options `accessControl` de `react-native-keychain`.

Erreurs fréquentes : `if (biometricOk) showAccount()` sans clé ; logique « isAdmin » ou « isPremium » vérifiée seulement dans l'app ; jetons sans expiration ; mot de passe stocké pour « reconnecter » l'utilisateur ; OTP par SMS considéré comme résistant à l'hameçonnage.

Checklist AUTH

- ☐ OAuth2/OIDC + PKCE via navigateur système (pas de WebView de connexion).
- ☐ Toute autorisation vérifiée côté serveur, à chaque requête.
- ☐ Politique de mots de passe alignée sur NIST SP 800-63B-4 + limitation des essais.
- ☐ Biométrie liée à une clé (CryptoObject / SecAccessControl).
- ☐ Ré-authentification pour les opérations sensibles.
- ☐ Révocation des sessions côté serveur ; passkeys envisagées.

2.2.4 MASVS-NETWORK — communications réseau

Contrôle	Énoncé officiel (traduction)
MASVS-NETWORK-1	L'app sécurise tout le trafic réseau selon les bonnes pratiques actuelles.
MASVS-NETWORK-2	L'app effectue l'épinglage d'identité (pinning) pour tous les points de terminaison distants sous le contrôle du développeur.

- HTTPS partout, TLS 1.2 minimum (1.3 de préférence) côté serveur, HSTS sur les domaines web associés.
- Android : depuis Android 9 (API 28), le trafic en clair est désactivé par défaut ; ne le réactivez pas (`cleartextTrafficPermitted`), et ne faites jamais confiance aux CA ajoutées par l'utilisateur en production.
- iOS : App Transport Security (ATS) est actif par défaut ; n'ajoutez pas `NSAllowsArbitraryLoads`. Attention : ATS ne s'applique pas aux API réseau de bas niveau (Network framework, sockets) — c'est alors à vous de valider la connexion.
- Ne désactivez jamais la validation des certificats (`TrustManager` qui accepte tout, `HostnameVerifier` permissif, `badCertificateCallback => true` en Dart, `NSAllowsArbitraryLoads`). C'est l'erreur la plus fréquente laissée après le débogage.
- Aucune donnée sensible dans les URL (paramètres de requête journalisés par les proxys et serveurs) ; utilisez le corps ou les en-têtes.

Android — Network Security Config

```
<!-- res/xml/network_security_config.xml -->
<network-security-config>
    <base-config cleartextTrafficPermitted="false">
        <trust-anchors>
            <certificates src="system" />          <!-- pas de CA "user" -->
        </trust-anchors>
    </base-config>
    <!-- Pinning optionnel (voir nuances) : toujours une broche de secours -->
    <domain-config>
        <domain includeSubdomains="true">api.exemple.ca</domain>
        <pin-set expiration="2027-06-30">
            <pin digest="SHA-256">base64ClePubliquePrincipale=</pin>
            <pin digest="SHA-256">base64CleDeSecours=</pin>
        </pin-set>
    </domain-config>
    <debug-overrides>                               <!-- builds debug seulement -->
        <trust-anchors><certificates src="user" /></trust-anchors>
    </debug-overrides>
</network-security-config>
<!-- AndroidManifest : android:networkSecurityConfig="@xml/network_security_config" -->
```

iOS — NSPinnedDomains

```
<!-- Info.plist : pinning déclaratif ATS (iOS 14+) -->
<key>NSAppTransportSecurity</key>
<dict>
  <key>NSPinnedDomains</key>
  <dict>
    <key>api.exemple.ca</key>
    <dict>
      <key>NSIncludesSubdomains</key><true/>
      <key>NSPinnedCAIdentities</key>
      <array>
        <dict><key>SPKI-SHA256-BASE64</key><string>base64...=</string></dict>
        <dict><key>SPKI-SHA256-BASE64</key><string>base64Secours...=</string></dict>
      </array>
    </dict>
  </dict>
</dict>
```

Certificate pinning : les nuances

Le pinning est exigé par MASVS-NETWORK-2 pour les profils de haute sécurité, mais il est à double tranchant :

- **Android Developers** indique que le pinning de certificats « n'est pas recommandé pour les applications Android », car un changement de configuration serveur (nouvelle CA, nouvelle clé) peut couper toutes les

versions installées jusqu'à une mise à jour. Si vous le faites malgré tout : plusieurs broches de secours dont au moins une clé entièrement sous votre contrôle, et une période d'expiration suffisamment courte. La même page rappelle qu'une date d'expiration peut aussi permettre de contourner le pinning une fois passée.

- Épinglez la **clé publique (SPKI)** d'une CA intermédiaire ou de votre propre clé plutôt que le certificat feuille, qui change à chaque renouvellement (souvent tous les 90 jours avec Let's Encrypt).
- Le pinning protège contre une CA compromise ou une CA malveillante installée par l'utilisateur ; il **n'empêche pas** un attaquant qui contrôle son propre appareil d'analyser votre trafic (Frida/objection le désactivent en quelques secondes). Ce n'est donc pas une protection de l'API : l'API doit rester sûre même si le trafic est lu.
- Prévoyez un plan opérationnel : inventaire des clés épinglées, procédure de rotation, alerte avant l'expiration, et mécanisme de mise à jour forcée (MASVS-CODE-2).
- Préférez la configuration déclarative (Network Security Config, NSPinnedDomains) au code maison. En Flutter/RN, privilégiez des solutions qui s'appuient sur ces mécanismes natifs ou des bibliothèques maintenues.

Checklist NETWORK

- ☐ Aucune exception cleartext/ATS en production (vérifier le manifeste et l'Info.plist).
- ☐ Aucun TrustManager/HostnameVerifier/callback de certificat permissif.
- ☐ CA utilisateur ignorées en release ; `debug-overrides` réservé au debug.
- ☐ Décision écrite sur le pinning (oui/non, SPKI, broches de secours, plan de rotation).
- ☐ Pas de données sensibles dans les URL ni dans les en-têtes journalisés.

2.2.5 MASVS-PLATFORM — interaction avec la plateforme

Contrôle	Énoncé officiel (traduction)
MASVS-PLATFORM-1	L'app utilise les mécanismes de communication inter-processus (IPC) de manière sécurisée.
MASVS-PLATFORM-2	L'app utilise les WebViews de manière sécurisée.
MASVS-PLATFORM-3	L'app utilise l'interface utilisateur de manière sécurisée.

- **IPC Android** : `android:exported="false"` par défaut pour activités, services, receivers et providers ; protégez ceux qui doivent être exportés par des permissions de signature ; utilisez des intents explicites ; `PendingIntent.FLAG_IMMUTABLE` ; `FileProvider` au lieu d'URI `file://`.
- **Liens profonds** : traitez chaque lien comme une entrée non fiable (validation des paramètres, pas d'action sensible sans confirmation). Préférez les **App Links** vérifiés (Android) et les **Universal Links** (iOS) aux schémas personnalisés, que d'autres apps peuvent revendiquer. N'envoyez jamais de jeton dans un lien profond (mise en garde de la doc React Native).
- **WebViews** : désactivez JavaScript s'il n'est pas nécessaire, interdisez l'accès aux fichiers (`allowFileAccess=false`), chargez uniquement des domaines en liste blanche via HTTPS, n'exposez pas de pont natif (`addJavascriptInterface`, `JavascriptChannel`, `postMessage`) à du contenu non maîtrisé. Pas de WebView pour la connexion OAuth.
- **UI** : masquez les champs sensibles, désactivez les suggestions/autocorrection sur les champs secrets, protégez contre le tapjacking (`filterTouchesWhenObscured`), limitez la copie vers le presse-papiers, masquez les données sensibles dans les notifications.
- Permissions : demandez le minimum, au moment où la fonctionnalité en a besoin, avec une explication claire (lien direct avec PRIVACY-1).

Android / Kotlin — WebView

```
// Android : WebView restreinte
```

```
webView.settings.apply {
    javaScriptEnabled = false          // true seulement si indispensable
    allowFileAccess = false
    allowContentAccess = false
}
webView.webViewClient = object : WebViewClient() {
    override fun shouldOverrideUrlLoading(v: WebView, r: WebResourceRequest): Boolean {
        val host = r.url.host ?: return true
        return host != "aide.exemple.ca"       // bloque tout autre domaine
    }
}
```

Checklist PLATFORM

- ☐ Tous les composants Android non nécessaires en `exported=false`.
- ☐ Liens profonds validés ; App Links / Universal Links pour les flux importants.
- ☐ WebViews : JS/fichiers désactivés par défaut, liste blanche de domaines, pas de pont natif exposé.
- ☐ Champs sensibles masqués, protection tapjacking, notifications sans données sensibles.

2.2.6 MASVS-CODE — qualité du code et mises à jour

Contrôle	Énoncé officiel (traduction)
MASVS-CODE-1	L'app exige une version de plateforme à jour.
MASVS-CODE-2	L'app dispose d'un mécanisme pour imposer les mises à jour.
MASVS-CODE-3	L'app n'utilise que des composants logiciels sans vulnérabilités connues.
MASVS-CODE-4	L'app valide et assainit toutes les entrées non fiables.

- **Versions** : fixez un `minSdk`/deployment target raisonnable et suivez les exigences des stores. Depuis le 31 août 2026, Google Play exige que les nouvelles apps et mises à jour ciblent Android 16 (API 36) ; depuis le 28 avril 2026, Apple exige Xcode 26 et le SDK iOS 26 pour tout téléversement dans App Store Connect.
- **Mise à jour forcée** : Android propose l'API In-App Updates (mode « immediate ») ; sur iOS, il n'y a pas d'équivalent système — implémentez un contrôle de version minimale renvoyé par votre backend qui bloque les versions vulnérables.
- **Dépendances** : inventaire (SBOM), analyse SCA automatisée (Dependabot/Renovate, OWASP Dependency-Check, `npm audit`, `flutter pub outdated`), mises à jour régulières, retrait des bibliothèques abandonnées. Voir 2.6.
- **Entrées** : validez tout ce qui vient du réseau, des liens profonds, des intents, du presse-papiers, des fichiers, des QR codes. Requêtes SQL paramétrées (Room, SQLite avec arguments liés), pas de désérialisation d'objets arbitraires, protection contre le « Zip Path Traversal ».
- Validation côté serveur obligatoire ; la validation côté client n'est qu'un confort d'UX.

Android / Kotlin — Room

```
// Room : requêtes paramétrées (jamais de concaténation de chaînes)
@Query("SELECT * FROM note WHERE owner_id = :ownerId AND title LIKE :q")
suspend fun search(ownerId: String, q: String): List<NoteEntity>
```

Checklist CODE

- ☐ minSdk / deployment target et target SDK conformes aux exigences des stores.
- ☐ Mécanisme de version minimale (backend) + In-App Updates Android.
- ☐ SCA en CI, SBOM généré, aucune vulnérabilité critique connue au moment de la release.

☐ Toutes les entrées non fiables validées ; requêtes paramétrées.

2.2.7 MASVS-RESILIENCE — résistance à la rétro-ingénierie et à la falsification

Contrôle	Énoncé officiel (traduction)
MASVS-RESILIENCE-1	L'app valide l'intégrité de la plateforme.
MASVS-RESILIENCE-2	L'app met en œuvre des mécanismes anti-falsification.
MASVS-RESILIENCE-3	L'app met en œuvre des mécanismes anti-analyse statique.
MASVS-RESILIENCE-4	L'app met en œuvre des techniques anti-analyse dynamique.

Ces contrôles relèvent de la **défense en profondeur** : ils ralentissent un attaquant mais ne remplacent jamais les contrôles serveur. Adoptez-les selon le risque (fraude, triche, contenu payant, données réglementées).

- **Attestation côté serveur (le plus utile)** : Android **Play Integrity API** (vérifie que la requête vient de votre app authentique installée par Google Play sur un appareil certifié ; le verdict est vérifié par votre serveur) et Apple **App Attest** (`DCAppAttestService`, clé matérielle dans la Secure Enclave certifiée par Apple ; assertions signées jointes aux requêtes sensibles). Apple rappelle que l'app ne peut pas se vérifier elle-même, car une app compromise peut falsifier le résultat : la décision doit être prise par le serveur.
- **Obfuscation** : Android R8 (`isMinifyEnabled = true`, recommandé par Google pour la performance et l'obfuscation) ; Flutter `--obfuscate --split-debug-info` (renomme les symboles, ne chiffre pas les ressources, ne protège pas des secrets) ; JavaScript React Native : Hermes compile en bytecode, ce qui n'est pas une protection forte. Conservez les fichiers de symboles pour désobfusquer les plantages.
- **Détection root/jailbreak, débogueur, hooks (Frida), émulateur** : utile pour réduire les risques ou dégrader les fonctions, mais contournable. Ne bloquez pas aveuglément (faux positifs) ; remontez plutôt un signal au serveur qui décide.
- **Anti-falsification** : vérification de signature/intégrité via l'attestation plutôt que des contrôles locaux faciles à « patcher ».
- Pour les besoins élevés (banque, paiement), envisagez une solution RASP commerciale reconnue, évaluée selon les profils de test MASTG.

Android — R8

```
// build.gradle.kts (module app)
android {
    buildTypes {
        release {
            isMinifyEnabled = true        // R8 : réduction + obfuscation
            isShrinkResources = true
            proguardFiles(getDefaultProguardFile("proguard-android-optimize.txt"),
                          "proguard-rules.pro")
        }
    }
}
```

Flutter — obfuscation

```
flutter build appbundle --obfuscate --split-debug-info=build/symbols/android
flutter build ipa       --obfuscate --split-debug-info=build/symbols/ios
```

iOS / Swift — App Attest

```
// iOS : App Attest (schéma simplifié)
import DeviceCheck
let service = DCAppAttestService.shared
guard service.isSupported else { /* repli prévu côté serveur */ return }
let keyId = try await service.generateKey()                // stocker keyId
```

```
let challenge = try await api.fetchChallenge()              // nonce serveur unique
let attestation = try await service.attestKey(keyId,
                     clientDataHash: Data(SHA256.hash(data: challenge)))
try await api.registerAttestation(keyId: keyId, attestation: attestation)
// Ensuite : service.generateAssertion(keyId, clientDataHash:) pour chaque requête sensible
```

Checklist RESILIENCE

- ☐ Évaluation du risque : RESILIENCE requis ? (oui si fraude/paiement/contenu payant/données réglementées).
- ☐ Play Integrity et App Attest vérifiés côté serveur pour les endpoints sensibles.
- ☐ R8 activé / Flutter obfusqué ; fichiers de symboles archivés en CI.
- ☐ Détection root/jailbreak/Frida comme signal envoyé au serveur, pas comme seule barrière.
- ☐ Aucune logique de sécurité critique qui dépend uniquement du client.

2.2.8 MASVS-PRIVACY — vie privée

Contrôle	Énoncé officiel (traduction)
MASVS-PRIVACY-1	L'app minimise l'accès aux données et ressources sensibles.
MASVS-PRIVACY-2	L'app empêche l'identification de l'utilisateur.
MASVS-PRIVACY-3	L'app est transparente sur la collecte et l'utilisation des données.
MASVS-PRIVACY-4	L'app offre à l'utilisateur le contrôle de ses données.

- Minimisation : ne collectez que le nécessaire ; préférez les sélecteurs système (Photo Picker Android, PHPicker iOS) aux permissions larges, la localisation approximative à la précise.
- Identifiants : pas d'identifiants matériels persistants ; identifiants pseudonymes propres à l'app ; respect d'App Tracking Transparency (iOS) pour le suivi entre apps.
- Transparence : déclarations exactes dans la section **Sécurité des données** de Google Play et les **détails de confidentialité** de l'App Store ; fichier `PrivacyInfo.xcprivacy` (privacy manifest) pour votre app et vos SDK ; politique de confidentialité claire.
- Contrôle : consentement granulaire, paramètres de confidentialité les plus protecteurs par défaut, export des données, **suppression du compte depuis l'app** (exigée par la règle 5.1.1(v) des App Review Guidelines d'Apple si l'app permet la création de compte).
- SDK tiers (analytics, publicité, crash) : inventaire, finalité, configuration minimale, prise en compte dans les déclarations et dans l'ÉFVP (voir 2.8).

Checklist PRIVACY

- ☐ Inventaire des données personnelles (quoi, pourquoi, où, combien de temps, avec qui).
- ☐ Permissions minimales, demandées en contexte.
- ☐ Déclarations Play Data safety / App Privacy et privacy manifest à jour.
- ☐ Suppression de compte et export des données disponibles dans l'app.
- ☐ Fonctions de localisation/profilage désactivées par défaut (Loi 25).

2.3 Sécurité du backend et des API (OWASP API Security Top 10 2023)

Votre application mobile n'est qu'un client parmi d'autres : n'importe qui peut appeler votre API directement avec curl ou Postman. La majorité des fuites de données « mobiles » viennent en réalité de l'API.

Risque (2023)	Ce que ça veut dire	Mesure concrète
API1 Broken Object Level Authorization	Accès à l'objet d'un autre utilisateur en changeant un identifiant (`/orders/123` → `/orders/124`).	Vérifier dans chaque handler que l'objet appartient à l'utilisateur du jeton ; identifiants non devinables (UUID) en complément, jamais à la place.
API2 Broken Authentication	Jetons mal validés, brute force, réinitialisation faible.	Fournisseur d'identité reconnu ; validation stricte des JWT (signature, `iss`, `aud`, `exp`) ; limitation de débit sur /login et /reset.
API3 Broken Object Property Level Authorization	Exposition excessive de champs ou « mass assignment ».	DTO de sortie explicites ; liste blanche des champs modifiables.
API4 Unrestricted Resource Consumption	Coûts ou déni de service (SMS, courriels, calculs).	Rate limiting par utilisateur/IP/appareil, quotas, pagination, taille max des requêtes.
API5 Broken Function Level Authorization	Accès à des fonctions d'administration.	Contrôle de rôle côté serveur ; API admin séparée.
API6 Unrestricted Access to Sensitive Business Flows	Automatisation abusive (achats, inscriptions).	Détection d'abus, attestation (Play Integrity/App Attest), CAPTCHA adaptatif.
API7 Server Side Request Forgery	Le serveur récupère une URL fournie par le client.	Liste blanche de destinations ; pas d'accès au réseau interne.
API8 Security Misconfiguration	CORS permissif, messages d'erreur verbeux, en-têtes manquants.	Configuration durcie, erreurs génériques côté client, journalisation côté serveur.
API9 Improper Inventory Management	Anciennes versions d'API oubliées, endpoints de debug exposés.	Inventaire des versions, désactivation des anciennes, OpenAPI à jour.
API10 Unsafe Consumption of APIs	Confiance aveugle envers des API tierces.	Valider et assainir les réponses des tiers, délais d'expiration, TLS.

Backend — exemple illustratif

```
// Exemple (Node/Express) : contrôle d'autorisation au niveau objet (API1)
app.get('/api/notes/:id', requireAuth, async (req, res) => {
  const note = await db.notes.findById(req.params.id);
  if (!note || note.ownerId !== req.user.sub) {
    return res.status(404).end();          // ne révèle pas l'existence
  }
  res.json(toNoteDto(note));               // DTO : champs explicites (API3)
});
```

Checklist API

- ☐ Autorisation objet + fonction vérifiée sur chaque endpoint (tests automatisés dédiés).
- ☐ JWT validés : signature, algorithme attendu, `iss`, `aud`, `exp`.
- ☐ Rate limiting et quotas sur l'authentification, les SMS/courriels et les endpoints coûteux.
- ☐ DTO de sortie, liste blanche d'entrée, erreurs génériques.
- ☐ Inventaire des versions d'API ; anciennes versions désactivées.

2.4 Gestion des secrets : jamais de clés dans l'app

Tout ce qui est dans le binaire est public : chaînes de caractères, `BuildConfig`, `Info.plist`, `.env` empaqueté, `--dart-define`, `react-native-config`, ressources. La documentation React Native le dit explicitement : ne stockez jamais de clé d'API sensible dans le code de l'app ; la documentation Flutter rappelle que stocker des secrets dans une app est une mauvaise pratique et que l'obfuscation ne les protège pas.

Type de valeur	Peut aller dans l'app ?	Où la mettre
URL d'API, identifiant client OAuth public, clé publique de pinning	Oui (non secrètes)	Configuration de build par environnement
Clé d'API d'un service tiers facturé (cartes, IA, SMS, paiement)	**Non**	Backend « proxy » (fonction serverless ou API) qui détient la clé et applique authentification + quotas
Clé Firebase/Google restreinte (par app, par API)	Oui, si restreinte et protégée par App Check/règles de sécurité	Fichier de config + restrictions dans la console
Secret client OAuth, clé de signature JWT, mot de passe BD	**Jamais**	Gestionnaire de secrets serveur (Vault, AWS/GCP/Azure Secrets Manager)
Clés de signature de l'app (keystore, certificats Apple)	**Jamais dans le dépôt**	Play App Signing ; secrets chiffrés de la CI ; coffre-fort
Jetons de l'utilisateur	Oui, à l'exécution	Keychain / Keystore (voir STORAGE)

- Analyse de secrets dans le dépôt et la CI (gitleaks, GitHub secret scanning, trufflehog) + hook pré-commit.
- `.env`, `google-services.json` de prod, `*.jks`, `*.p12`, `*.mobileprovision` dans `.gitignore` selon votre politique.
- Rotation immédiate de tout secret ayant fuité — le supprimer de l'historique Git ne suffit pas.

2.5 OAuth 2.0 / OpenID Connect + PKCE et gestion des jetons

RFC 8252 (BCP 212) impose que les applications natives **n'utilisent pas de user-agent intégré** (WebView) pour les requêtes d'autorisation, et utilisent le navigateur externe : Custom Tabs sur Android, `ASWebAuthenticationSession` sur iOS. Elle décrit trois types de redirection : schéma d'URI privé (ex. `ca.exemple.app:/oauth2redirect`), redirection `https` revendiquée (App Links / Universal Links, la plus sûre) et interface de bouclage (loopback, surtout pour le bureau). Les serveurs d'autorisation doivent prendre en charge PKCE pour ces clients publics.

Flux recommandé (Authorization Code + PKCE, RFC 7636) :

1. L'app génère un `code_verifier` aléatoire (43 à 128 caractères) et calcule `code_challenge = BASE64URL(SHA256(code_verifier))` (méthode `S256`, obligatoire côté serveur selon la RFC).
2. Elle ouvre le navigateur système sur `/authorize` avec `code_challenge`, `code_challenge_method=S256`, un `state` aléatoire et, pour OIDC, un `nonce`.
3. L'utilisateur s'authentifie (mot de passe, passkey, MFA) chez le fournisseur d'identité.
4. Redirection vers l'app avec `code` + `state` ; l'app vérifie `state`.
5. L'app échange `code` + `code_verifier` sur `/token` ; un code intercepté par une autre app est inutilisable sans le verifier.
6. Elle reçoit access token (courte durée), refresh token et ID token (OIDC) dont elle valide `iss`, `aud`, `exp`, `nonce`.

Kotlin — PKCE

```
// Génération PKCE (Kotlin) — les bibliothèques AppAuth le font pour vous
val bytes = ByteArray(32).also { SecureRandom().nextBytes(it) }
val b64 = Base64.getUrlEncoder().withoutPadding()
val codeVerifier  = b64.encodeToString(bytes)                    // 43 caractères
val codeChallenge = b64.encodeToString(
    MessageDigest.getInstance("SHA-256").digest(codeVerifier.toByteArray(Charsets.US_ASCII)))
```

React Native — OAuth + PKCE

```
// React Native : react-native-app-auth (enveloppe AppAuth-iOS / AppAuth-Android)
```

```
import { authorize } from 'react-native-app-auth';
const result = await authorize({
  issuer: 'https://auth.exemple.ca',
  clientId: 'mobile-app',                       // client public : pas de secret
  redirectUrl: 'ca.exemple.app:/oauth2redirect',
  scopes: ['openid', 'profile', 'offline_access'],
  usePKCE: true,                                // valeur par défaut
});
await Keychain.setGenericPassword('refresh_token', result.refreshToken);
```

Équivalents : **AppAuth-Android** et **AppAuth-iOS** (bibliothèques de référence de l'OpenID Foundation), `ASWebAuthenticationSession` (iOS), et en Flutter des plugins qui s'appuient sur AppAuth (par ex. `flutter_appauth`). Les SDK de fournisseurs (Auth0, Okta, Firebase Auth, Microsoft Entra / MSAL, Google Identity / Credential Manager) implémentent ces mêmes recommandations.

Règles pour les jetons

Sujet	Règle
Pas de secret client	Une app mobile est un client public : un secret embarqué dans une app distribuée ne peut pas être considéré comme confidentiel (RFC 8252, section 8.5). N'embarquez aucun `client_secret`.
Pas de flux implicite	Le flux implicite est « NOT RECOMMENDED » pour les apps natives car il ne peut pas être protégé par PKCE (RFC 8252, section 8.2) ; RFC 9700 indique que les clients ne devraient pas l'utiliser.
Access token	Durée courte (de l'ordre de quelques minutes), en mémoire de préférence, jamais dans les logs ni les URL.
Refresh token	Stocké dans Keychain/Keystore ; révocable. RFC 9700 exige que les refresh tokens des clients publics soient liés à l'émetteur (« sender-constrained », p. ex. DPoP) ou soumis à rotation (nouveau jeton à chaque usage, réutilisation détectée côté serveur).
Déconnexion	Révoquer le refresh token côté serveur, effacer le stockage local, fermer la session du fournisseur si nécessaire.
Validation côté API	Signature, algorithme attendu (refuser `none`), `iss`, `aud`, `exp`, portée (`scope`).
Renouvellement	Un seul rafraîchissement simultané (mutex/intercepteur) pour éviter les courses entre requêtes.

2.6 Chaîne d'approvisionnement, dépendances (SCA) et CI/CD sécurisé

Le risque M2 (Inadequate Supply Chain Security) couvre les bibliothèques, SDK tiers, outils de build et pipelines. Mesures :

- **Inventaire et verrouillage** : fichiers de verrouillage versionnés (`gradle.lockfile` ou version catalog, `Package.resolved`, `pubspec.lock`, `package-lock.json`/`yarn.lock`) ; SBOM (CycloneDX ou SPDX) généré à chaque release.
- **SCA automatisée** : Dependabot ou Renovate ; OWASP Dependency-Check ; `npm audit` ; OSV-Scanner (couvre Maven, npm, Pub, SwiftPM…). Bloquez la fusion en cas de vulnérabilité critique exploitable.
- **Choix des dépendances** : maintenance active, éditeur vérifié (pub.dev « verified publisher », npm provenance), licence compatible, nombre minimal de dépendances transitives ; méfiance envers les paquets au nom proche d'un paquet connu (typosquatting).
- **SAST** : Android Lint (règles de sécurité), CodeQL (Kotlin/Java, Swift, JavaScript/TypeScript), Semgrep, analyse statique MobSF du binaire produit.

- **Pipeline** : runners éphémères, secrets de CI à portée minimale, protection des branches (revue obligatoire, statut CI vert), actions tierces épinglées par SHA de commit, signature effectuée dans la CI ou via Play App Signing, artefacts et symboles conservés.
- **Signature** : Play App Signing (Google conserve la clé de signature de l'app ; vous gardez une clé d'importation) ; certificats et profils Apple gérés via App Store Connect / outils comme fastlane match avec stockage chiffré.

GitHub Actions — exemple

```
# .github/workflows/ci.yml — squelette illustratif (adapter les versions)
name: ci
on: [pull_request]
permissions: { contents: read }
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@<SHA-épinglé>
        with: { fetch-depth: 0 }
      - name: Secrets dans le dépôt
        uses: gitleaks/gitleaks-action@<SHA-épinglé>
      - name: Lint + tests unitaires
        run: ./gradlew lint detekt testDebugUnitTest   # ou: flutter analyze && flutter test
      - name: Vulnérabilités des dépendances
        run: osv-scanner scan --recursive .
      - name: Build release + analyse MobSF (job séparé, serveur MobSF interne)
        run: ./gradlew bundleRelease
```

2.7 Tests de sécurité

Combinez tests automatisés en CI et tests manuels guidés par le MASTG avant chaque version majeure. NIST SP 800-163 Rev. 1 décrit un processus de « vetting » : définir les exigences de sécurité de l'organisation, tester l'app (outils automatisés + analyse), évaluer les résultats et décider de l'approbation.

Outil	Type	Usage typique
MobSF (Mobile Security Framework)	Statique + dynamique, Android/iOS	Analyse automatisée d'APK/AAB/IPA : permissions, manifestes, secrets, configuration réseau, bibliothèques. Intégrable en CI via son API.
Frida	Instrumentation dynamique	Hooker des fonctions à l'exécution : vérifier que la biométrie, la détection root ou le pinning ne sont pas triviaux à contourner.
objection	Boîte à outils basée sur Frida	Explorer la Keychain/le stockage, désactiver le pinning pour tester l'API.
jadx / apktool	Décompilation Android	Vérifier ce qu'un attaquant voit : secrets, endpoints, logique sensible.
Burp Suite / mitmproxy / OWASP ZAP	Proxy d'interception	Tester l'API (BOLA, injection, rate limiting) et le comportement TLS.
Ghidra	Rétro-ingénierie native	Analyse de bibliothèques natives (.so) et binaires iOS.
Android Lint, SwiftLint, CodeQL, Semgrep	SAST	Détection précoce dans le code source.

- Testez sur un appareil rooté/jailbreaké ou un émulateur dédié, jamais sur votre téléphone personnel.
- Écrivez des **tests automatisés d'autorisation** sur l'API (utilisateur A ne lit pas les données de B).
- Avant le lancement d'une app sensible : test d'intrusion par un tiers indépendant, sur la base du MASVS/MASTG.

2.8 Conformité vie privée au Québec : Loi 25 et LPRPDE

Si votre entreprise exerce au Québec, la **Loi sur la protection des renseignements personnels dans le secteur privé**, modernisée par la **Loi 25** (adoptée en 2021, en vigueur par étapes les 22 septembre 2022, 2023 et 2024), s'applique. La **LPRPDE** fédérale s'applique notamment aux renseignements qui franchissent les frontières provinciales ou nationales dans le cadre d'activités commerciales et repose sur 10 principes relatifs à l'équité dans le traitement de l'information (responsabilité, détermination des fins, consentement, limitation de la collecte, limitation de l'utilisation/communication/conservation, exactitude, mesures de sécurité, transparence, accès individuel, possibilité de porter plainte). Faites valider le champ d'application exact par un ou une juriste.

Exigence (source : CAI)	Traduction concrète dans l'app et le backend
Responsable de la protection des renseignements personnels ; titre et coordonnées publiés sur le site Web	Page « Confidentialité » accessible depuis l'app et la fiche store.
Politiques et pratiques de gouvernance (conservation, destruction, rôles, plaintes)	Durées de conservation par type de donnée, purges automatiques, processus de plainte documenté.
Politique de confidentialité en termes simples et clairs pour toute collecte par un moyen technologique	Lien dans l'app (écran d'inscription et paramètres) + avis de modification.
Évaluation des facteurs relatifs à la vie privée (ÉFVP) pour tout projet de développement de système d'information ou de prestation électronique de services impliquant des renseignements personnels, et avant toute communication hors Québec	Faire l'ÉFVP dès la conception de l'app ; la plupart des hébergeurs/SaaS (cloud, analytics, crash) traitent des données hors Québec.
Consentement manifeste, libre, éclairé, à des fins précises, demandé pour chaque fin	Écrans de consentement séparés (analytics, marketing, localisation), pas de cases précochées, retrait possible.
Paramètres de confidentialité au plus haut niveau par défaut	Partage, visibilité publique, analytics non essentielles : désactivés par défaut.
Technologies permettant d'identifier, de localiser ou de profiler : informer au préalable et ne pas activer par défaut	Localisation et profilage en opt-in explicite.
Transparence au moment de la collecte : fins, moyens, droits, tiers, communication possible hors Québec	Texte « juste à temps » au moment de la demande de permission.
Renseignements sensibles (médicaux, biométriques…) ; biométrie : consentement exprès et déclaration préalable à la CAI	La biométrie système (Face ID/Touch ID/BiometricPrompt) reste sur l'appareil ; si vous constituez une banque biométrique côté serveur, obligations supplémentaires (divulgation à la CAI au plus tard 60 jours avant sa mise en service).
Mineurs de moins de 14 ans : consentement du titulaire de l'autorité parentale	Contrôle d'âge si votre public peut inclure des enfants.
Décision fondée exclusivement sur un traitement automatisé : informer et permettre des observations	À prévoir si l'app utilise un algorithme de décision (scoring, IA).
Incidents de confidentialité : registre ; avis à la CAI et aux personnes en cas de risque de préjudice sérieux	Journalisation de sécurité, plan de réponse aux incidents, registre tenu.
Droit à la portabilité (depuis le 22 septembre 2024) : format technologique structuré et couramment utilisé	Fonction « Exporter mes données » (JSON/CSV).

Enjeu — La CAI peut imposer des sanctions administratives pécuniaires pouvant atteindre 10 millions de dollars ou 2 % du chiffre d'affaires mondial (source : CAI, « Principaux changements… »). Des amendes pénales plus élevées sont aussi prévues par la loi. La vie privée doit donc être traitée comme une exigence de conception, pas comme une formalité de fin de projet.

Checklist Loi 25 / LPRPDE

- ☐ ÉFVP réalisée et documentée avant la mise en production (inclut les SDK et hébergements hors Québec).
- ☐ Responsable PRP désigné ; politique de confidentialité publiée et liée dans l'app.
- ☐ Consentements séparés par finalité, retrait possible, paramètres protecteurs par défaut.
- ☐ Localisation/profilage désactivés par défaut et activables explicitement.
- ☐ Export des données (portabilité) et suppression du compte dans l'app.
- ☐ Durées de conservation définies et purges automatisées.
- ☐ Registre et procédure de gestion des incidents de confidentialité.

2.9 Checklist de sécurité consolidée (avant chaque release)

☐ **STORAGE** : secrets/jetons uniquement en Keychain/Keystore ; aucune donnée sensible dans logs, sauvegardes, caches, captures.

☐ **CRYPTO** : API de plateforme, AES-GCM/SHA-256/ECDSA, clés matérielles non exportables, aucune clé en dur.

☐ **AUTH** : OAuth2/OIDC + PKCE via navigateur système ; autorisation côté serveur ; biométrie liée à une clé ; ré-auth des opérations sensibles.

☐ **NETWORK** : HTTPS uniquement, aucune exception cleartext/ATS, aucune validation de certificat désactivée ; décision pinning documentée.

☐ **PLATFORM** : composants non exportés, liens profonds validés, WebViews restreintes, UI protégée.

☐ **CODE** : target SDK conforme, mise à jour forcée possible, SCA verte, entrées validées.

☐ **RESILIENCE** : R8/obfuscation, Play Integrity / App Attest vérifiés côté serveur si le risque le justifie.

☐ **PRIVACY** : minimisation, déclarations stores et privacy manifest exacts, consentement, export et suppression.

☐ **API** : BOLA/BFLA testés, rate limiting, JWT validés, versions inventoriées.

☐ **Secrets** : scan de secrets vert, aucun secret dans le binaire (vérifié avec jadx/MobSF).

☐ **CI/CD** : revue obligatoire, actions épinglées, signature protégée, SBOM archivé.

☐ **Tests** : analyse MobSF sans constat critique ; tests MASTG ciblés ; test d'intrusion pour les apps sensibles.

☐ **Loi 25** : ÉFVP à jour pour les nouvelles fonctionnalités et les nouveaux fournisseurs.

3. Clean code

Les principes ci-dessous sont développés dans les ouvrages de référence cités en bibliographie (Robert C. Martin, Martin Fowler, Hunt et Thomas, Gamma et al.). Ils sont résumés ici avec nos propres mots et appliqués au contexte mobile.

3.1 Nommage

- Des noms qui révèlent l'intention : `remainingTrialDays` plutôt que `d` ; `fetchInvoices()` plutôt que `getData()`.
- Vocabulaire du domaine métier, cohérent dans l'app, l'API et la base (langage omniprésent de Domain-Driven Design, Evans).
- Méthodes = verbes (`makePayment()`), propriétés = noms (`inProgressTopicSelection`), flux = `get{Modèle}Stream()` — conventions proposées par le guide d'architecture Android ; classes nommées selon leur rôle (`HomeViewModel`, `UserRepository`, `ClientApiService`) — convention du guide Flutter.
- Respectez les conventions de la langue : Kotlin coding conventions, Swift API Design Guidelines, Effective Dart, conventions TypeScript/ESLint.

3.2 Fonctions et classes

- Petites fonctions qui font une seule chose à un seul niveau d'abstraction ; peu de paramètres (regroupez-les dans un type).
- Pas d'effets de bord cachés : une fonction `isValid()` ne doit pas écrire en base.
- Séparez commande (modifie l'état) et requête (renvoie une valeur).
- Données immuables par défaut (`val`, `let`, `final`, `readonly`, `data class`/`struct`/`freezed`) : moins de bogues de concurrence et un état d'UI prévisible.
- Pas de « God class » : un ViewModel de 1 500 lignes signale qu'il faut extraire des cas d'usage ou des composants.

3.3 SOLID appliqué au mobile

Principe	Idée	Exemple mobile
S — Responsabilité unique	Une classe a une seule raison de changer.	`LoginViewModel` gère l'état de l'écran ; `AuthRepository` gère les jetons ; `TokenStorage` gère la Keychain.
O — Ouvert/fermé	Étendre sans modifier le code existant.	Nouveau moyen de paiement = nouvelle implémentation de `PaymentMethod`, sans toucher au `when`/`switch` central.
L — Substitution de Liskov	Une implémentation doit respecter le contrat de son abstraction.	`FakeUserRepository` (tests) et `OfflineFirstUserRepository` se comportent de façon interchangeable.
I — Ségrégation des interfaces	Plusieurs petites interfaces plutôt qu'une grosse.	`NoteReader` et `NoteWriter` au lieu d'un `NoteManager` à 20 méthodes.
D — Inversion des dépendances	Le code de haut niveau dépend d'abstractions.	Le ViewModel dépend de l'interface `UserRepository`, injectée ; jamais de `Retrofit`/`Dio`/`fetch` directement dans l'UI.

3.4 DRY, KISS, YAGNI

- **DRY** (The Pragmatic Programmer) : chaque connaissance a une représentation unique — règles de validation, mapping d'erreurs, constantes d'API. Attention : deux codes qui se ressemblent mais changent pour des raisons différentes ne sont pas une duplication de connaissance.
- **KISS** : la solution la plus simple qui fonctionne. Pas de Clean Architecture complète pour une app de trois écrans.
- **YAGNI** : n'implémentez pas une fonctionnalité ou une abstraction « au cas où ». Ajoutez la couche domain ou un nouveau module quand le besoin apparaît (les guides Android et Flutter la qualifient d'ailleurs d'optionnelle/conditionnelle).

3.5 Gestion des erreurs

- Modélisez les erreurs attendues comme des valeurs : `sealed interface Result`/`sealed class` (Kotlin), `Result<T, E>` et `throws` typés (Swift), classes `Result` scellées (Dart 3 `sealed`), union discriminée en TypeScript.
- Traduisez les erreurs à la frontière de chaque couche : `HttpException` → `NetworkError.Unavailable` dans la couche data → message utilisateur dans l'UI.
- Ne jamais avaler une exception (`catch {}` vide) ; ne jamais afficher de trace technique à l'utilisateur ; journaliser sans données personnelles.
- Gérez explicitement : hors-ligne, délai dépassé, 401 (rafraîchir puis déconnecter), 403, 429 (attente exponentielle), 5xx.
- Annulation correcte des tâches asynchrones (coroutines structurées, `Task` Swift, `CancelToken` Dio, `AbortController`).

Kotlin — état d'écran exhaustif

```
sealed interface LoginUiState {
    data object Idle : LoginUiState
    data object Loading : LoginUiState
    data class Error(val message: UserMessage) : LoginUiState
    data class Success(val userName: String) : LoginUiState
}
```

3.6 Tests

- Pyramide : beaucoup de tests unitaires (ViewModels, cas d'usage, repositories, mappers), des tests d'intégration (base locale, client HTTP avec serveur simulé), quelques tests UI/bout en bout sur les parcours critiques.
- Préférez les **fakes** aux mocks (recommandation explicite des guides Android et Flutter) : un `FakeUserRepository` en mémoire est plus robuste qu'une cascade de `when(...).thenReturn(...)`.
- Outils : JUnit + kotlinx-coroutines-test + Turbine + Compose UI test + Macrobenchmark (Android) ; XCTest / Swift Testing + XCUITest (iOS) ; `flutter_test`, tests de widgets, `integration_test` (Flutter) ; Jest + React Native Testing Library + Detox ou Maestro (RN).
- Tests de sécurité automatisés : autorisation API, absence de secrets, configuration réseau (lint).
- Visez une couverture significative des couches domain/data plutôt qu'un pourcentage global.

3.7 Revues de code

- Pull requests petites (idéalement < 400 lignes modifiées), avec description, captures d'écran et lien vers le ticket.

- Automatiser ce qui peut l'être (format, lint, tests, SCA) pour concentrer la revue humaine sur la conception, la lisibilité et la sécurité.
- Grille de revue : lisibilité, nommage, tests, gestion d'erreurs, données sensibles (logs, stockage), autorisation, accessibilité, performances, migration de base de données.
- Seul ? Relisez votre PR le lendemain, utilisez la checklist et les outils d'analyse ; demandez des revues ponctuelles à des pairs pour les parties sensibles (auth, paiement).

3.8 Linters et formateurs par stack

Stack	Formatage	Analyse statique / lint	Sécurité
Android / Kotlin	ktlint ou ktfmt	Android Lint, detekt	Android Lint (règles de sécurité), CodeQL, MobSF
iOS / Swift	swift-format	SwiftLint, analyseur Xcode (Analyze)	CodeQL (Swift), MobSF
Flutter / Dart	`dart format`	`dart analyze` + `flutter_lints` ou `very_good_analysis`	MobSF sur l'APK/IPA, OSV-Scanner
React Native / TS	Prettier	ESLint (config React Native), TypeScript `strict`	`npm audit`, CodeQL (JS/TS), MobSF

Activez ces outils dans l'IDE, en hook pré-commit et en CI (échec du build en cas d'erreur).

4. Architecture

4.1 Les patterns comparés

Pattern	Principe	Forces	Faiblesses	Quand l'utiliser
MVVM	La Vue observe l'état exposé par un ViewModel ; les actions de l'utilisateur appellent des méthodes du ViewModel.	Recommandé officiellement par Android et Flutter ; naturel avec Compose, SwiftUI, Flutter, hooks React ; simple à tester.	Peut dériver vers des ViewModels trop gros ; plusieurs flux d'état désynchronisés si l'on n'impose pas un état unique.	Par défaut, presque partout.
MVI	État d'écran unique et immuable ; l'UI émet des intentions ; un réducteur produit le nouvel état (flux unidirectionnel strict).	Prévisibilité, débogage (« time travel »), état cohérent, très testable.	Plus de code ; courbe d'apprentissage ; risque de sur-ingénierie sur écrans simples.	Écrans complexes avec beaucoup d'interactions ; équipes habituées à Redux/Bloc/TCA.
Clean Architecture	Couches concentriques : domain (entités, cas d'usage, interfaces) au centre, data et presentation à l'extérieur ; dépendances orientées vers le domaine.	Domaine indépendant des frameworks ; logique métier réutilisable et testable ; frontières nettes.	Verbosité (mappers, cas d'usage « passe-plat ») ; lourd pour une petite app.	Logique métier riche, longue durée de vie, plusieurs développeurs.
Architecture modulaire	Découpage en modules Gradle / Swift Packages / paquets Dart / workspaces : `core:*` et `feature:*`.	Builds incrémentaux plus rapides, frontières imposées par le compilateur, propriété par équipe.	Coût de configuration ; graphe de dépendances à maîtriser.	Dès que l'app dépasse quelques fonctionnalités ou que l'équipe grandit.

Ces approches ne s'excluent pas : la pratique recommandée par les guides officiels combine **MVVM avec flux unidirectionnel (UDF)** dans la couche UI, une **couche data à base de repositories** comme source unique de vérité, une **couche domain optionnelle**, et une **modularisation par fonctionnalité** quand la taille le justifie.

4.2 Recommandation principale

Recommandation — Adoptez une architecture en couches « Clean Architecture pragmatique » : UI (MVVM + flux unidirectionnel, un état d'écran immuable par écran) → Domain optionnel (cas d'usage, uniquement pour la logique réutilisée ou complexe) → Data (repositories comme source unique de vérité, avec sources locale et distante). Injection de dépendances par constructeur, interfaces sur les repositories, tests avec des fakes, et modularisation par fonctionnalité dès que l'app grandit. C'est précisément le modèle que recommandent les guides officiels d'Android (Strongly recommended : couches UI et data, UDF, repositories, DI ; domaine « recommandé dans les grosses apps ») et de Flutter (Strongly recommend : couches UI/data, MVVM, repository, UDF, DI ; domaine « conditionnel »).

Pourquoi ce choix :

- Il est aligné sur la documentation officielle des deux principaux écosystèmes ; vous bénéficiez donc des exemples, outils et bibliothèques prévus pour cette structure (Hilt, Room, Compose, go_router, provider…).
- Il est progressif : on commence avec deux couches (UI + data) et on ajoute la couche domain et des modules quand la complexité augmente (YAGNI), sans réécriture.
- La séparation des couches sert aussi la sécurité : la couche data centralise le stockage sécurisé, l'ajout de jetons, la validation et la gestion des erreurs ; l'UI ne touche jamais directement au réseau ni au stockage.

- Il est le plus facile à tester (ViewModels et repositories testables sans appareil).
- MVI pur et Clean Architecture « complète » restent des options pour des écrans ou des domaines qui le justifient — pas un point de départ obligatoire.

Déclinaison par stack

Stack	UI / état	DI	Données / offline	Navigation
Android (Kotlin)	Jetpack Compose ; ViewModel exposant un `StateFlow<UiState>` (`stateIn(WhileSubscribed(5000))`), collecté avec `collectAsStateWithLifecycle`	Hilt (ou DI manuelle pour une petite app)	Room + DataStore ; WorkManager pour la synchronisation ; Retrofit/Ktor	Navigation Compose / Navigation 3, activité unique
iOS (Swift)	SwiftUI ; ViewModel `@Observable` (Observation, iOS 17+) ou `ObservableObject` ; `async/await`	Injection par initialiseur ; conteneur léger si besoin	SwiftData ou Core Data (ou GRDB) ; `URLSession`	`NavigationStack` piloté par l'état
Flutter (Dart)	Views + ViewModels (`ChangeNotifier`) selon le guide officiel ; alternatives : Riverpod, Bloc (style MVI)	`provider` (recommandé par le guide Flutter) ; Riverpod ou get_it en option	Repositories + services ; drift ou sqflite ; modèles immuables `freezed`	`go_router`
React Native (TS)	Composants fonctionnels ; hooks personnalisés jouant le rôle de ViewModel ; état serveur : TanStack Query ; état client : Zustand ou Redux Toolkit	Props/Context ; modules injectables pour les services	Repositories TS ; SQLite (expo-sqlite / op-sqlite) ou WatermelonDB ; MMKV pour les préférences non sensibles	React Navigation ou Expo Router

Et le choix de la stack ?

Option	Arguments pour un développeur web	Points d'attention
React Native (+ Expo)	Réutilise React, TypeScript et l'écosystème npm ; transition la plus douce depuis le web ; une base de code pour iOS et Android.	Dépendances natives tierces à surveiller (SCA) ; performances à profiler sur les animations lourdes ; JavaScript lisible dans le bundle.
Flutter	Rendu cohérent sur les deux plateformes ; guide d'architecture officiel très prescriptif ; outillage intégré (tests, lints, obfuscation).	Nouveau langage (Dart, facile pour un dev JS/TS) ; intégrations natives via canaux de plateforme.
Natif (Kotlin + Swift)	Accès complet et immédiat aux API de sécurité (Keystore, Secure Enclave, App Attest, Play Integrity) et aux nouveautés OS ; meilleures performances.	Deux bases de code et deux langages ; coût double pour une personne seule.

Pour un développeur web seul qui vise iOS et Android, **React Native avec Expo ou Flutter** sont généralement les choix les plus efficaces ; le natif se justifie si l'app dépend fortement de capacités matérielles ou de sécurité avancées. Dans tous les cas, l'architecture recommandée ci-dessus s'applique.

4.3 Structures de dossiers types

Android — multi-modules (inspiré de l'app exemple « Now in Android »)

```
app/                        -> point d'entrée, navigation, DI racine (Hilt)
core/
  model/                    -> modèles partagés (Kotlin pur)
  domain/                   -> cas d'usage (optionnel), interfaces
```

```
  data/                       -> repositories (implémentations)
  database/                   -> Room (DAO, entités)
  network/                    -> Retrofit/Ktor, intercepteur d'auth
  security/                   -> Keystore, stockage des jetons
  designsystem/               -> thème, composants Compose
  testing/                    -> fakes, règles de test
feature/
  login/   (LoginScreen.kt, LoginViewModel.kt, LoginUiState.kt)
  notes/   (NotesScreen.kt, NotesViewModel.kt, ...)
```

iOS — Swift Packages locaux

```
MonApp/                       -> cible app : App.swift, composition racine (DI)
Packages/
  Domain/      Sources/Domain/        (entités, protocoles de repository, cas d'usage)
  Data/        Sources/Data/          (APIClient, repositories, persistance)
  Security/    Sources/Security/      (Keychain, App Attest)
  DesignSystem/Sources/DesignSystem/
  Features/
    Login/     Sources/Login/         (LoginView.swift, LoginViewModel.swift)
    Notes/     Sources/Notes/
Tests/ (un target de tests par package)
```

Flutter — structure proposée par le guide d'architecture officiel

```
lib/
  main.dart / main_development.dart / main_staging.dart
  config/                     -> injection de dépendances (providers)
  routing/                    -> go_router
  domain/models/              -> modèles immuables (freezed)
  domain/use_cases/           -> optionnel
  data/
    repositories/<feature>/   -> abstract class + implémentations (remote, fake)
    services/                 -> clients API, stockage sécurisé, BD locale
  ui/
    core/                     -> thème et widgets partagés (pas "widgets/")
    <feature>/view_models/
    <feature>/widgets/
test/  (même arborescence)   testing/ (fakes)
```

React Native — par fonctionnalité

```
src/
  app/                        -> navigation, providers, bootstrap
  features/
    auth/   (screens/, hooks/useLogin.ts, components/, auth.repository.ts)
    notes/
  domain/                     -> types métier, règles pures (TS sans dépendance RN)
  data/
    api/                      -> client HTTP, intercepteurs (jetons, erreurs)
    db/                       -> SQLite / WatermelonDB
    secure/                   -> wrapper Keychain/SecureStore
  ui/                         -> design system
  test/                       -> fakes, utilitaires
```

4.4 Injection de dépendances

- Injection par **constructeur** partout où c'est possible (recommandation Android) ; pas de singletons globaux accessibles de partout (recommandation Flutter).
- Une **racine de composition** unique (module Hilt, `App` SwiftUI, `MultiProvider` Flutter, provider React) qui assemble les implémentations réelles ; les tests y substituent des fakes.
- Portées explicites : singleton pour le client HTTP et la base, portée écran pour les ViewModels.

- Les composants de sécurité (stockage de jetons, client attesté) sont injectés : on peut ainsi les remplacer en test sans affaiblir la configuration de production.

Android — Hilt + ViewModel

```
@Module @InstallIn(SingletonComponent::class)
abstract class DataModule {
    @Binds abstract fun bindNotes(impl: OfflineFirstNotesRepository): NotesRepository
}

@HiltViewModel
class NotesViewModel @Inject constructor(repo: NotesRepository) : ViewModel() {
    val uiState: StateFlow<NotesUiState> = repo.getNotesStream()
        .map { NotesUiState.Success(it) }
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), NotesUiState.Loading)
}
```

iOS — ViewModel @Observable avec injection par initialiseur

```
@Observable
final class NotesViewModel {
    private(set) var state: NotesState = .loading
    private let repository: NotesRepository           // protocole
    init(repository: NotesRepository) { self.repository = repository }

    func load() async {
        do { state = .loaded(try await repository.notes()) }
        catch { state = .failed(UserMessage(error)) }
    }
}
```

Flutter — ViewModel (ChangeNotifier) + Result

```
class NotesViewModel extends ChangeNotifier {
  NotesViewModel({required NotesRepository repository}) : _repo = repository;
  final NotesRepository _repo;
  NotesState _state = const NotesState.loading();
  NotesState get state => _state;

  Future<void> load() async {
    final result = await _repo.getNotes();         // Result<List<Note>>
    _state = switch (result) {
      Ok(:final value)   => NotesState.loaded(value),
      Error(:final error) => NotesState.failed(error),
    };
    notifyListeners();
  }
}
```

4.5 Gestion d'état

- Distinguez **état d'UI** (formulaire, sélection, chargement), **état applicatif** (session, préférences) et **état serveur** (données mises en cache).
- Un état d'écran **unique et immuable** exposé par le ViewModel (Android : `uiState: StateFlow` ; Flutter : modèles immuables) ; l'UI ne fait que l'afficher et émettre des événements (UDF).
- Android recommande de ne pas envoyer d'« événements » ponctuels du ViewModel vers l'UI : traitez l'événement dans le ViewModel et reflétez le résultat dans l'état.
- N'y mettez jamais de secrets si l'état est persisté (piège Redux-persist signalé par la doc React Native).

4.6 Offline-first

Le guide Android « Build an offline-first app » décrit le modèle de référence, transposable à toutes les stacks :

- La **base locale est la source de vérité** : l'UI observe la base ; le réseau sert à la synchroniser.
- Lectures : affichage immédiat depuis le cache, rafraîchissement en arrière-plan.
- Écritures : enregistrement local + file d'attente de synchronisation (WorkManager sur Android, `BGTaskScheduler` sur iOS, `workmanager` en Flutter, tâches en arrière-plan Expo/RN), avec reprise et attente exponentielle.
- Conflits : stratégie explicite (dernier écrit gagne, versionnement, fusion par champ) décidée avec le backend, qui reste l'arbitre.
- Sécurité : chiffrez la base si elle contient des données sensibles (SQLCipher ou clé protégée par Keystore/Keychain), purgez-la à la déconnexion, revalidez toutes les écritures côté serveur (le client hors-ligne peut avoir été modifié).

4.7 Tests par couche

Couche	Quoi tester	Comment
Domain (cas d'usage, règles)	Règles métier, calculs, validations	Tests unitaires purs, très rapides, sans framework
Data (repositories, mappers, sources)	Mapping DTO ↔ modèle, cache, stratégie offline, erreurs réseau	Fakes de sources ; serveur HTTP simulé (MockWebServer, `URLProtocol` personnalisé, `http` MockClient, MSW) ; base en mémoire
Presentation (ViewModels)	Transitions d'état (Loading → Success/Error), réactions aux événements	Fakes de repositories ; `runTest` + Turbine ; tests asynchrones Swift ; `flutter_test` ; Jest + hooks testing
UI	Rendu des états, accessibilité, navigation	Compose UI tests, XCUITest/snapshots, tests de widgets, React Native Testing Library
Bout en bout	Parcours critiques (inscription, connexion, paiement)	Maestro, Detox, `integration_test`, Espresso/XCUITest, sur appareils réels en CI (Firebase Test Lab, appareils physiques)
Sécurité	Autorisation API, stockage, configuration réseau	Tests d'API automatisés, MobSF en CI, tests MASTG manuels

5. Cycle de développement complet

Phase	Activités clés	Livrables sécurité / qualité
1. Cadrage et planification	Objectifs, utilisateurs, fonctionnalités MVP, choix de stack, inventaire des données personnelles, choix des fournisseurs (auth, hébergement, analytics).	Profil de sécurité visé (contrôles MASVS retenus) ; début de l'ÉFVP (Loi 25) ; registre des données.
2. Conception	Architecture (section 4), contrats d'API (OpenAPI), maquettes, modèle de menaces STRIDE.	Diagramme de flux de données, menaces et contre-mesures, exigences de sécurité dans le backlog.
3. Développement	Branches courtes, PR revues, tests, lint, SCA.	Pipeline CI vert, checklist de PR, aucun secret dans le dépôt.
4. Vérification	Tests automatisés, tests manuels MASTG ciblés, MobSF, tests d'API, bêta.	Rapport de tests, constats corrigés ou acceptés formellement.
5. Publication	Signature, fiches stores, déclarations de confidentialité, déploiement progressif.	Data safety / App Privacy / privacy manifest exacts ; notes de version.
6. Exploitation	Monitoring, réponse aux incidents, mises à jour.	Tableaux de bord, alertes, procédure d'incident (registre Loi 25).

5.1 Modélisation des menaces (STRIDE)

STRIDE (méthode issue de Microsoft) classe les menaces en six catégories. Appliquez-la sur un diagramme de flux de données simple : utilisateur → app → API → base de données / services tiers, en marquant les frontières de confiance (l'appareil, Internet, votre cloud). Pour chaque flux et chaque composant, posez les questions ci-dessous ; consignez les menaces, leur priorité et la contre-mesure dans le backlog. Revoyez le modèle à chaque fonctionnalité importante.

Catégorie	Propriété violée	Exemples mobiles	Contre-mesures
S — Spoofing (usurpation)	Authentification	Faux client appelant l'API ; app malveillante interceptant un schéma d'URI ; hameçonnage.	OAuth + PKCE, passkeys, App Links/Universal Links, attestation Play Integrity / App Attest.
T — Tampering (falsification)	Intégrité	App reconditionnée ; requêtes modifiées ; base locale modifiée sur appareil rooté.	Validation serveur, signature, attestation, TLS, contrôles d'intégrité.
R — Repudiation (répudiation)	Non-répudiation	Utilisateur niant une transaction.	Journaux d'audit côté serveur horodatés, signature des opérations sensibles (clé matérielle).
I — Information disclosure (divulgation)	Confidentialité	Jetons dans les logs, sauvegardes, captures ; BOLA ; trafic intercepté.	STORAGE/NETWORK du MASVS, autorisation objet, minimisation.
D — Denial of service (déni de service)	Disponibilité	Envoi massif de SMS d'OTP ; requêtes coûteuses ; app bloquée par un pinning expiré.	Rate limiting, quotas, plan de rotation des broches, disjoncteurs.
E — Elevation of privilege (élévation de privilèges)	Autorisation	Appel d'une fonction admin ; composant Android exporté ; pont JavaScript de WebView.	BFLA côté serveur, `exported=false`, WebViews restreintes, moindre privilège.

Les quatre questions du Threat Modeling Manifesto résument la démarche : sur quoi travaillons-nous ? qu'est-ce qui peut mal tourner ? que faisons-nous pour y remédier ? avons-nous fait du bon travail ?

5.2 CI/CD

7. **À chaque PR** : format + lint, tests unitaires, scan de secrets, SCA, build debug, SAST.
8. **Sur la branche principale** : build release signé (clé en secret CI ou Play App Signing), tests UI/E2E sur appareils, analyse MobSF du binaire, génération du SBOM, archivage des symboles (mapping R8, symboles Dart, dSYM).
9. **Distribution de test** : piste de test interne / fermée Google Play, TestFlight, ou Firebase App Distribution.
10. **Publication** : promotion manuelle (approbation) vers la production avec déploiement progressif.
11. Outils courants : GitHub Actions, GitLab CI, Bitrise, Codemagic (Flutter), EAS Build/Submit (Expo), fastlane (automatisation signature/téléversement).

5.3 Publication sur les stores

Sujet	Google Play	App Store
Exigence SDK (au 10 oct. 2026)	Depuis le 31 août 2026 : nouvelles apps et mises à jour doivent cibler Android 16 (API 36) (exceptions Wear OS, Automotive, TV, XR).	Depuis le 28 avril 2026 : build avec Xcode 26 et SDK iOS 26 ; depuis le 9 septembre 2026 : cible iOS 13 ou plus.
Tests avant production	Pistes interne, fermée, ouverte. Comptes personnels créés après le 13 novembre 2023 : test fermé avec au moins 12 testeurs inscrits en continu pendant 14 jours avant de demander l'accès à la production.	TestFlight (testeurs internes et externes) ; App Review avant publication.
Confidentialité	Section Sécurité des données (Data safety) ; politique de confidentialité.	Détails de confidentialité (App Privacy) ; privacy manifest `PrivacyInfo.xcprivacy` ; App Tracking Transparency ; suppression de compte dans l'app (règle 5.1.1(v)).
Signature	Play App Signing (clé d'application gérée par Google, clé d'importation chez vous).	Certificats de distribution et profils de provisionnement (App Store Connect).
Déploiement progressif	Déploiements par étapes (staged rollouts) en pourcentage, suspendables.	Phased release : mise à jour automatique répartie sur 7 jours, suspendable.
Mises à jour	API In-App Updates (flexible / immediate).	Pas de mécanisme système de mise à jour forcée : contrôle de version minimale via votre backend.

Veille — Les exigences des stores changent chaque année (généralement au printemps pour Apple, en août pour Google Play). Consultez « Upcoming Requirements » (Apple) et « Target API level requirements » (Play Console) avant chaque planification de release.

5.4 Monitoring et observabilité

- Plantages et ANR : Firebase Crashlytics ou Sentry ; Android vitals (Play Console) ; Xcode Organizer et MetricKit (iOS).
- Performances : démarrage, rendu, réseau (Firebase Performance Monitoring, Macrobenchmark en CI, Instruments).
- Sécurité côté serveur : journaux d'authentification, taux d'échecs, verdicts Play Integrity/App Attest, anomalies de débit, alertes.
- Vie privée : masquage des données personnelles dans les rapports (scrubbing), consentement préalable pour l'analytique non essentielle, durées de rétention des journaux, fournisseurs pris en compte dans l'ÉFVP.

- Indicateurs produit : taux de sessions sans plantage, adoption des versions (pour décider quand forcer une mise à jour).

5.5 Mises à jour et maintenance

- Cadence régulière (par ex. mensuelle) de mise à jour des dépendances, plus correctifs de sécurité immédiats.
- Suivi annuel des nouvelles versions d'Android et d'iOS (bêtas dès leur sortie) et des changements de comportement.
- Rotation planifiée des clés de pinning, des certificats et des secrets serveur.
- Procédure d'incident : détection → confinement (révocation de jetons, désactivation d'une fonctionnalité via feature flag, version minimale) → correctif → analyse d'impact → avis à la CAI et aux personnes si risque de préjudice sérieux → inscription au registre.
- Fin de vie : désactivez proprement les anciennes versions d'API et d'app (OWASP API9).

6. Bibliographie et liens

Liens vérifiés le 10 octobre 2026.

6.1 Standards et référentiels de sécurité

- OWASP MAS — MASVS (v2.1.0) : https://mas.owasp.org/MASVS/
- OWASP MASTG : https://mas.owasp.org/MASTG/
- OWASP MASWE : https://mas.owasp.org/MASWE/
- OWASP MAS Checklist : https://mas.owasp.org/checklists/
- OWASP MAS Testing Profiles : https://mas.owasp.org/Profiles/
- OWASP Mobile Top 10 (2024) : https://owasp.org/www-project-mobile-top-10/2023-risks/
- OWASP API Security Top 10 (2023) : https://owasp.org/API-Security/editions/2023/en/0x11-t10/
- OWASP Threat Modeling Cheat Sheet : https://cheatsheetseries.owasp.org/cheatsheets/Threat_Modeling_Cheat_Sheet.html
- Threat Modeling Manifesto : https://www.threatmodelingmanifesto.org/
- Microsoft — Threat Modeling Tool, catégories STRIDE : https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats
- NIST SP 800-163 Rev. 1, Vetting the Security of Mobile Applications (2019) : https://csrc.nist.gov/pubs/sp/800/163/r1/final
- NIST SP 800-63B-4, Authentication and Authenticator Management (2025) : https://csrc.nist.gov/pubs/sp/800/63/b/4/final
- IETF RFC 8252 — OAuth 2.0 for Native Apps (BCP 212) : https://www.rfc-editor.org/rfc/rfc8252
- IETF RFC 7636 — Proof Key for Code Exchange (PKCE) : https://www.rfc-editor.org/rfc/rfc7636
- IETF RFC 9700 — Best Current Practice for OAuth 2.0 Security : https://www.rfc-editor.org/rfc/rfc9700
- OpenID Connect Core 1.0 : https://openid.net/specs/openid-connect-core-1_0.html

6.2 Android

- Security — vue d'ensemble et risques courants : https://developer.android.com/privacy-and-security/security-tips
- Cryptography : https://developer.android.com/privacy-and-security/cryptography
- Android Keystore system : https://developer.android.com/privacy-and-security/keystore
- Network Security Configuration : https://developer.android.com/privacy-and-security/security-config
- Security with network protocols (TLS, pinning) : https://developer.android.com/privacy-and-security/security-ssl
- Play Integrity API : https://developer.android.com/google/play/integrity/overview
- Biometric authentication : https://developer.android.com/identity/sign-in/biometric-auth
- Guide to app architecture : https://developer.android.com/topic/architecture
- Recommendations for Android architecture : https://developer.android.com/topic/architecture/recommendations
- Build an offline-first app : https://developer.android.com/topic/architecture/data-layer/offline-first
- Guide to Android app modularization : https://developer.android.com/topic/modularization
- Dependency injection with Hilt : https://developer.android.com/training/dependency-injection/hilt-android
- In-app updates : https://developer.android.com/guide/playcore/in-app-updates
- Target API level requirements (Play) : https://developer.android.com/google/play/requirements/target-sdk

- Exigences de test pour les nouveaux comptes personnels : https://support.google.com/googleplay/android-developer/answer/14151465
- Now in Android (app exemple) : https://github.com/android/nowinandroid

6.3 Apple

- Apple Platform Security Guide : https://support.apple.com/guide/security/welcome/web
- Keychain Services : https://developer.apple.com/documentation/security/keychain-services
- Accessing keychain items with Face ID or Touch ID : https://developer.apple.com/documentation/localauthentication/accessing-keychain-items-with-face-id-or-touch-id
- Preventing insecure network connections (ATS) : https://developer.apple.com/documentation/security/preventing-insecure-network-connections
- NSPinnedDomains : https://developer.apple.com/documentation/bundleresources/information-property-list/nsapptransportsecurity/nspinneddomains
- Establishing your app's integrity (App Attest) : https://developer.apple.com/documentation/devicecheck/establishing-your-app-s-integrity
- Privacy manifest files : https://developer.apple.com/documentation/bundleresources/privacy-manifest-files
- App Review Guidelines : https://developer.apple.com/app-store/review/guidelines/
- Upcoming Requirements : https://developer.apple.com/news/upcoming-requirements/
- Phased release : https://developer.apple.com/help/app-store-connect/update-your-app/release-a-version-update-in-phases/

6.4 Flutter et React Native

- Flutter — Guide to app architecture : https://docs.flutter.dev/app-architecture
- Flutter — Architecture recommendations : https://docs.flutter.dev/app-architecture/recommendations
- Flutter — Obfuscate Dart code : https://docs.flutter.dev/deployment/obfuscate
- Flutter — Security : https://docs.flutter.dev/security
- flutter_secure_storage : https://pub.dev/packages/flutter_secure_storage
- React Native — Security : https://reactnative.dev/docs/security
- react-native-app-auth : https://github.com/FormidableLabs/react-native-app-auth
- react-native-keychain : https://github.com/oblador/react-native-keychain
- Expo SecureStore : https://docs.expo.dev/versions/latest/sdk/securestore/
- AppAuth (OpenID Foundation) : https://appauth.io/

6.5 Outils de test et de chaîne d'approvisionnement

- MobSF : https://github.com/MobSF/Mobile-Security-Framework-MobSF
- Frida : https://frida.re/
- objection : https://github.com/sensepost/objection
- jadx : https://github.com/skylot/jadx
- mitmproxy : https://mitmproxy.org/
- Ghidra : https://ghidra-sre.org/
- OWASP Dependency-Check : https://owasp.org/www-project-dependency-check/
- OSV-Scanner : https://google.github.io/osv-scanner/
- gitleaks : https://github.com/gitleaks/gitleaks
- CycloneDX (SBOM) : https://cyclonedx.org/

6.6 Vie privée — Québec et Canada

- CAI — Principaux changements (Loi 25) : https://www.cai.gouv.qc.ca/protection-renseignements-personnels/sujets-et-domaines-dinteret/principaux-changements-loi-25
- Loi sur la protection des renseignements personnels dans le secteur privé (RLRQ c. P-39.1) : https://www.legisquebec.gouv.qc.ca/fr/document/lc/P-39.1
- Gouvernement du Québec — Entrée en vigueur (22 sept. 2023) : https://www.quebec.ca/nouvelles/actualites/details/loi-25-entree-en-vigueur-de-dispositions-qui-rehaussent-la-protection-de-la-vie-privee-des-quebecois-50719
- Commissariat à la protection de la vie privée du Canada — principes de la LPRPDE : https://www.priv.gc.ca/fr/sujets-lies-a-la-protection-de-la-vie-privee/lois-sur-la-protection-des-renseignements-personnels-au-canada/la-loi-sur-la-protection-des-renseignements-personnels-et-les-documents-electroniques-lprpde/p_principle/

6.7 Livres de référence

Cités par titre, auteur et année uniquement. Procurez-vous-les par des canaux légitimes (éditeur, librairie, bibliothèque, O'Reilly Learning).

Titre	Auteur(s)	Année
Clean Code: A Handbook of Agile Software Craftsmanship	Robert C. Martin	2008
Clean Architecture: A Craftsman's Guide to Software Structure and Design	Robert C. Martin	2017
Refactoring: Improving the Design of Existing Code (2e édition)	Martin Fowler	2018 (1re éd. 1999)
The Pragmatic Programmer (édition du 20e anniversaire)	David Thomas, Andrew Hunt	2019 (1re éd. 1999)
Design Patterns: Elements of Reusable Object-Oriented Software	Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides (« Gang of Four »)	1994
Domain-Driven Design: Tackling Complexity in the Heart of Software	Eric Evans	2003
Mobile Application Security	Himanshu Dwivedi, Chris Clark, David Thiel	2010
The Mobile Application Hacker's Handbook	Dominic Chell, Tyrone Erasmus, Shaun Colley, Ollie Whitehouse	2015

Remarque — Les deux ouvrages de sécurité mobile (2010 et 2015) restent utiles pour la méthodologie d'attaque, mais leurs détails techniques sont antérieurs aux versions récentes d'Android et d'iOS : pour les contrôles à jour, le MASVS v2 et le MASTG font référence.

Annexe A. Correspondance indicative MASVS ↔ OWASP Mobile Top 10 2024

Correspondance établie par l'auteur du guide pour faciliter la lecture (elle n'est pas publiée telle quelle par l'OWASP).

Risque Mobile Top 10 2024	Groupes MASVS principalement concernés	Sections du guide
M1 Improper Credential Usage	AUTH, STORAGE	2.2.3, 2.4, 2.5
M2 Inadequate Supply Chain Security	CODE (CODE-3)	2.6
M3 Insecure Authentication/Authorization	AUTH	2.2.3, 2.3, 2.5
M4 Insufficient Input/Output Validation	CODE (CODE-4), PLATFORM	2.2.5, 2.2.6
M5 Insecure Communication	NETWORK	2.2.4
M6 Inadequate Privacy Controls	PRIVACY, STORAGE	2.2.8, 2.8
M7 Insufficient Binary Protections	RESILIENCE	2.2.7
M8 Security Misconfiguration	PLATFORM, NETWORK, CODE	2.2.4, 2.2.5
M9 Insecure Data Storage	STORAGE	2.2.1
M10 Insufficient Cryptography	CRYPTO	2.2.2